

Assignment 1: Tokenization, GPT-2 Analysis, and GPU Inference

Version 0.2 (Updated on 2026-02-24)

Large Language Models : A Hands-On Approach

Center for Continuing Education, IISc, Bangalore

Jan - May 2026

Instructors : Dr. Yoginder Negi, Bhuthesh R

Total Points: 40

Format: Individual Submission (Jupyter Notebook + PDF/Word/Markdown Report)

Environment: Google Colab (Free Tier T4 GPU)

Due Date: March 12, 2026 at 11:59 PM IST

Late Submission Penalty:

- Within due date : 0%
- Before March 31, 2026: 25%
- Before April 15, 2026: 50%
- Before May 1, 2026: 75%
- After May 1, 2026: 100%

Section 1: Tokenization Trade-offs (8 points)

Goal: Understand the trade-offs between different tokenization strategies.

Tasks

1. Implement two tokenizers from scratch:

- **Character-level:** Vocabulary = all unique characters (case-sensitive)
- **Whitespace:** Split on whitespace, with punctuation as separate tokens

2. Compare with Tiktoken (GPT-2 BPE):

- Tokenize this book (url : <https://www.gutenberg.org/cache/epub/11/pg11.txt>) with all three tokenizers
- Compute: **compression ratio** (characters / tokens), vocabulary size, and average token length

Deliverables

- Code for both tokenizers with `encode()` and `decode()` methods
 - **Bar chart:** Compare the three tokenizers on vocabulary size and compression ratio
 - **Analysis (~ 100 words):** Explain the trade-off between vocabulary size and sequence length. Why might a small vocabulary (char-level) lead to worse model performance despite preserving all information?
-

Section 2: Model Parameters : Inference Memory Architecture (16 points)

Goal:* Analyze the parameter distribution and activation memory requirements of GPT-2 models of varying sizes

Configurations

Use these exact hyperparameters from the OpenAI GPT-2 [Table 2](#) release:

Model	Layers (L)	d_{model}	Heads	d_{ff}	Vocab (V)	Max Seq (T)	Total Params
Small	12	768	12	3072	50,257	1024	~124M
Medium	24	1024	16	4096	50,257	1024	~355M
Large	36	1280	20	5120	50,257	1024	~774M

Task 1: Parameter Census (8 points)

Use FP32 (4 bytes/param) and FP16 (2 bytes/param).

Required table structure (verify your totals match the reference values exactly):

Component	Formula	Small (124M)	Medium (355M)	Large (774M)
Embeddings				
Token	$V \times d_{\text{model}}$	-	-	-
Positional	$T \times d_{\text{model}}$	-	-	-
Per Layer		$\times 12$	$\times 24$	$\times 36$
Attention QKV	$3 \times d_{\text{model}}^2$	-	-	-
Attention Out	d_{model}^2	-	-	-
MLP Up	$d_{\text{model}} \times d_{\text{ff}}$	-	-	-
MLP Down	$d_{\text{ff}} \times d_{\text{model}}$	-	-	-
LayerNorms ($\times 2$)	$2 \times d_{\text{model}}$	-	-	-
Total Per Layer		~7.1M	~12.6M	~19.7M
All Layers	-	-	-	-
TOTAL PARAMS	-	-	-	-
FP32 Memory	-	-	-	-
FP16 Memory	-	-	-	-

Task 2: Activation Memory Analysis (4 points)

For inference (forward pass only), calculate peak activation memory at $\text{batch_size} = 1$, $\text{sequence_length} = 1024$.

Account for the following tensors:

- Input embeddings: $B \times T \times d_{\text{model}}$
- Attention Q/K/V projections: $B \times L \times T \times d_{\text{model}}$ each
- Attention scores: $B \times L \times T \times T$: the $O(T^2)$ memory bottleneck

- MLP activations: $B \times T \times d_{\text{ff}}$
- LayerNorm inputs (retained for residual connections)

Calculate:

- Peak activation memory for the Small model (in MB)
- Ratio of activation memory to parameter memory for each model size
- At what model size do activations become negligible compared to parameters?

Task 3: Scaling Laws (4 points)

- Verify your parameter counts match the reference values ($\pm 0.01\%$):
 - Small: 124,439,808 params
 - Medium: 354,823,168 params
 - Large: 773,903,616 params
- Plot parameter count vs. FP16 memory for all three models on a log-log scale

Deliverables

- Completed parameter table with exact counts and memory totals in MB
- Activation memory calculation showing the $O(T^2)$ attention score block dominates ephemeral memory
- Scaling plot: parameter count (x-axis) vs. FP16 memory in MB (y-axis) on a log-log scale
- **Analysis (200 words):** “Why does the Medium model use $\sim 2.9\times$ the memory of Small, but Large only uses $\sim 2.2\times$ the memory of Medium, despite both adding 12 layers?” (*Hint: embedding matrices are constant size; only transformer blocks scale with L .*)

Key constraints:

- **No training components:** Exclude gradients, optimizer states, or backward-pass buffers
 - **Forward pass only:** Focus on what exists in GPU memory during `model.eval()` with `torch.no_grad()`
 - **Exact configs:** Use the table values above precisely
-

Section 3: From CPU to GPU : Inference Profiling & Optimization (16 points)

Environment: Google Colab (Free Tier) with T4 GPU **Goal:** Quantify the CPU vs. GPU performance gap

Pre-flight Check

Include the following cells at the top of your notebook and include its output in the submission:

```
import torch
print(f"PyTorch: {torch.__version__}")
print(f"CUDA Available: {torch.cuda.is_available()}")
if torch.cuda.is_available():
    print(f"Device: {torch.cuda.get_device_name(0)}")
    print(f"VRAM:
{torch.cuda.get_device_properties(0).total_memory / 1e9:.2f} GB")

!nvidia-smi --query-gpu=name,memory.total,memory.free --format=csv
```

Task 1: CPU Baseline (5 points)

Run inference on the CPU only (disable CUDA or use `map_location='cpu'`). Test all three model sizes with the following setup:

- **Input:** Batch size = 1, sequence length = 512 (any prompt or text from above book) generate 128 new tokens autoregressively
- **Metrics to collect:**
 - Cold-start latency (time to first token)
 - Steady-state latency (average latency over tokens 10–50)
 - Throughput (tokens/sec)
 - System RAM usage (`psutil` or `torch.cuda.memory_allocated()`)

Important: Use `torch.no_grad()` throughout. Run 2 warm-up forward passes before timing.

Deliverable : fill this table with measured values:

Model	First Token Latency	Throughput (tok/s)	RAM Used
Small 124M	-	-	-
Medium 355M	-	-	-

Model	First Token Latency	Throughput (tok/s)	RAM Used
Large 774M	-	-	-

Task 2: GPU Migration & Basic Acceleration (5 points)

Move each model to the T4 GPU (`model = model.cuda()`) and repeat the same generation task.

Measure:

- **Device transfer time:** Time moving parameters from CPU RAM -> GPU VRAM (*use `time.time()` before and after `.cuda()`*)
- **Inference throughput:** Same task as Task 1; use `torch.cuda.synchronize()` before and after generation
- **Peak VRAM usage:** `torch.cuda.max_memory_allocated()` after each run

Compute the **speedup ratio** (CPU time / GPU time) for each model size.

Think about this: Does speedup scale linearly with model size? Why does the Large model hit ~20–30× while Small only manages ~5–10×? (*Hint: fixed GPU overhead doesn't go away for small models.*)

Task 3: Optimization Experiments (4 points)

Apply the following optimizations to the Medium model (355M). For each, measure the change in tokens/sec and VRAM relative to the FP32 GPU baseline.

Option A: Mixed Precision (FP16)

```
model = model.half().cuda() # Convert weights to FP16
with torch.no_grad():
    outputs = model.generate(...)
```

- Measure speedup vs. FP32 (expect ~1.5–2× on T4 due to Tensor Cores)
- Check output quality: compute perplexity on a test sentence and confirm it stays within 1% of FP32

Option B: Batch Size Scaling

- Test batch sizes: [1, 2, 4, 8, 16] (stop if OOM)
- Plot throughput (tokens/sec) vs. batch size

- Mark the saturation point where adding more to the batch stops helping
- Compute per-token latency at each batch size and discuss the latency/throughput trade-off

Deliverables

- Before/after table for each optimization: FP32 baseline -> optimized (latency, throughput, VRAM)
- For Option B: throughput vs. batch size plot with saturation point annotated
- **Observation:** “At batch size X, we hit approximately Y% of the T4’s theoretical peak FLOPs”
- **One failure mode:** Show what happens when you try batch size 32 on the Large model - include the full OOM error message

Final Comparison Matrix (2 points)

Fill this in with your **actual measured values** (no placeholders):

Configuration	Device	Precision	Batch	Tok/ sec	VRAM (GB)	Speedup vs. CPU
Small 124M	CPU	FP32	1		N/A	1.0×
Small 124M	T4	FP32	1		~0.5	~8×
Small 124M	T4	FP16	1		~0.25	~12×
Medium 355M	T4	FP16	8		~1.5	~25×
Large 774M	T4	FP16	4		~3.2	~30×

Colab Constraints & Warnings

- **Session limits:** Colab will disconnect after idle time - if that happens, re-run everything from scratch. Keep all model definitions inside the notebook. If you hit OOM, don’t load all three models at once; free memory between runs:

```
del model_small
torch.cuda.empty_cache()
```

- **CPU baseline:** Do CPU tests before enabling GPU, or set `CUDA_VISIBLE_DEVICES=""` to force CPU-only mode.
- **Timing noise:** Do 3 warm-up iterations, then average over 5 timed runs.

Grading Rubric (Total: 40 points)

Section	Criterion	Points	Full Marks Requires
S1 (8)	Tokenizer Implementation	3	Both tokenizers pass round-trip test: <code>decode(encode(text)) == text</code>
	Tiktoken Comparison	2	Correct compression ratios; chart with labeled axes
	Analysis	3	Clear explanation of vocab size vs. sequence length trade-off with an information-theory connection
S2 (16)	Parameter Census	8	All cells filled with exact counts matching reference values ($\pm 0.1\%$); correct FP32/FP16 byte totals
	Activation Analysis	4	Correct $O(T^2)$ calculation; attention scores identified as dominant memory consumer; ratios for all three sizes
	Scaling Verification	4	Log-log plot with correct slope (~ 1.0); explanation of sub-linear scaling due to constant embedding matrices
S3 (16)	CPU Baseline	5	Correct timing methodology (warm-up + no_grad); all three models tested; OOM handling documented if applicable
	GPU Migration	5	Correct speedup ratios; transfer time measured; explanation of non-linear speedup across model sizes
	Optimizations	4	Two optimizations with quantitative before/after results; Option A verified

Section	Criterion	Points	Full Marks Requires
			within 1% perplexity if chosen
	Final Matrix	2	All cells filled with actual measured values

Penalties

- Missing `torch.cuda.synchronize()` in GPU timing code: **-2 points** (S3)
- Hardcoded values in the parameter table without accompanying formulas: **-5 points** (S2)
- Using non-standard configurations (e.g., $d_{\text{model}} = 512$ instead of 768 for Small): **-3 points per deviation** (S2)

Submission Checklist

- ☐ Jupyter notebook with all cells executed (*Runtime -> Run All*)
- ☐ PDF / Markdown / Word report containing all tables, plots, and analysis paragraphs
- ☐ Pre-flight check cell output confirming T4 GPU availability
- ☐ If you used AI assistance, include the prompt in the report and mark the code in colab as AI generated where applicable

How to submit

- Zip your notebook and report, name it as `Assignment1_<YourName>.zip` and send via email to `yoginder@iisc.ac.in` and cced to `bhuthesh.comp@gmail.com`